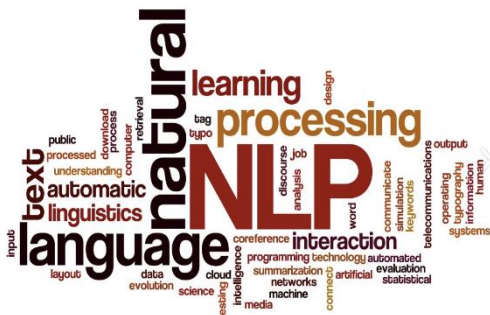
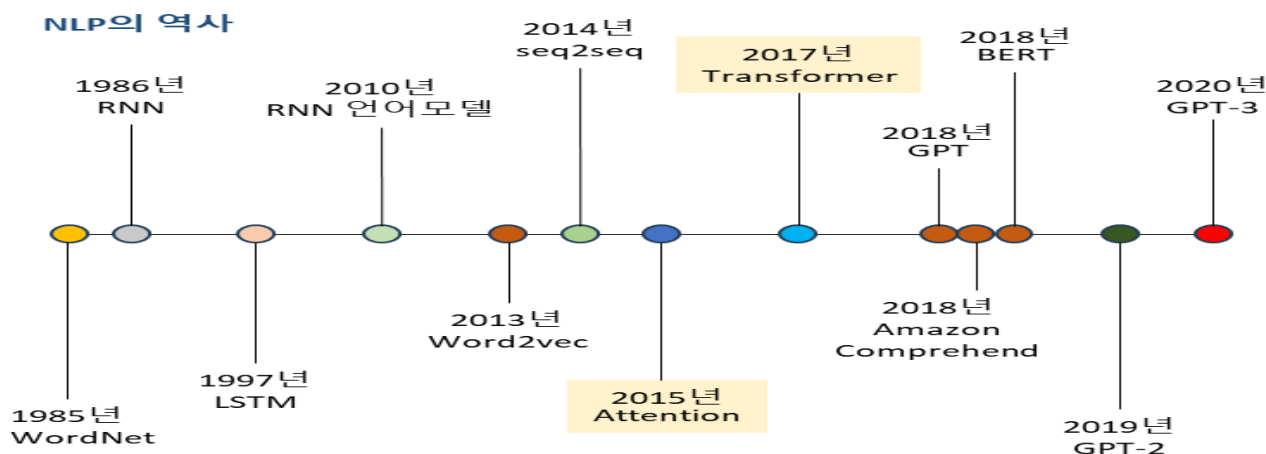


자연어 처리, NLP

Natural Language Processing, 줄여서 NLP는 인공지능의 한 분야로, **컴퓨터가 사람의 언어를 이해하고 처리하는 기술**을 말한다. 요약, 기계 번역, 감정분석, QA 등 많은 곳에서 응용된다. 인간의 언어는 **단어 간의 순서가 고려되는 시퀀셜 데이터**이기 때문에 딥러닝 분야에서 상대적으로 나중에 발전되어 왔지만, end to end 모델로 대체되면서 빠르게 발전되어 왔다.



사람의 말을 컴퓨터가 이해하도록 만들어서, 컴퓨터가 우리에게 도움이 되는 일을 수행하게 하는 것 외에도 대용량 텍스트에서 정보를 찾는 것부터 음성인식, 문서요약, 문서 분류, 이메일 필터, 감성분석 챗봇 등이 모두 자연어 처리 분야에 해당된다.



단어 임베딩(예: Word2Vec, GloVe)의 획기적인 발전으로 고차원 벡터 공간에서 단어 표현이 가능해졌으며 단어 간의 의미론적 관계를 포착할 수 있었다.

순환 신경망(RNN), 장단기 기억(LSTM) 네트워크 및 이후의 Transformer는 기계 번역 및 텍스트 생성과 같은 시퀀스 간 작업을 위한 강력한 모델로 등장했다. 트랜스포머(Transformer)는 구글이 자연어처리를 위해 2017년 발표한 모델로 현재 AI 분야의 혁신을 이끌고 있는 언어모델이다. 자연어처리 뿐만 아니라 컴퓨터 비전이나 음성 인식 등 다른 분야에도 적용되며 최고 수준의 성능을 기록하고 있다.

트랜스포머는 **어텐션을 통해 문장 길이에 대한 제약을 극복**하여 입력 문장과 생성 문장에 대한 이해를 넓혔다. 그리고 모든 연산 과정에서 입력 값과 가중치간 어마어마한 행렬 연산이 이뤄지는 데 이것을 모두 **병렬적으로 처리**함으로써 효율적인 연산이 가능하게 되었다. 이로 인해 대규모 데이터셋을 사전에 학습해 뛰어난 성능을 보이는 GPT (Generative Pre-trained Transformer)와 같은 대규모 언어모델의 기반이 되었다.

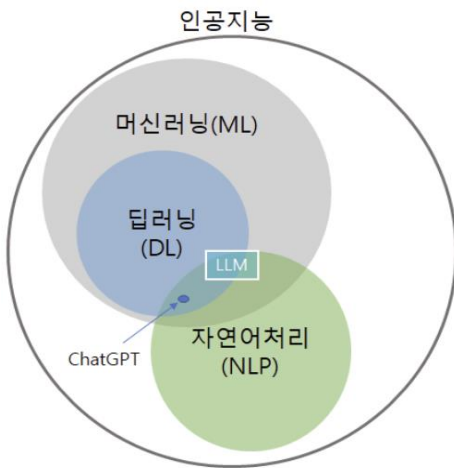
트랜스포머 모델은 자연어처리 뿐만 아니라 이미지 처리 분야에서도 좋은 결과를 내고 있다. 2020년 발표된 **Vision Transformer(ViT)**는 트랜스포머 모델을 비전 분야에 적용한 것으로, 입력 이미지를 패치로 분할해 트랜스포머의 인코더에 입력하고 어텐션을 사용해 입력 이미지의 다른 위치 간 상호의존성과 이미지의 글로벌 특징을 캡처할 수 있다.

구글은 2017년 **Attention is all you need**라는 논문을 통해 어텐션을, 보정을 위해 사용하는 것 뿐만 아니라 인코더와 디코더를 모두 어텐션으로 구현한 트랜스포머 모델을 소개했다. 트랜스포머 모델은 문장의 길이에 대한 제약은 물론 인코더가 인풋 문장을 보다 잘 이해하고 디코더가 자신이 앞서 생성한 단어들에 대해서도 보다 더 잘 이해할 수 있는 모델이 되었다. **문장 속 단어와 같은 순차 데이터(시퀀스) 내의 관계를 추적해 맥락과 의미를 학습하는 신경망이다.**

어텐션의 기본 아이디어는 인코더의 마지막 타임스텝에서 나온 수치 정보만으로는 부족하니 디코더에서 출력 단어를 예측하는 매 시점(time step)마다 인코더의 전체 입력 문장을 다시 한 번 참고한다는 것이다. 단, 전체 입력 문장의 단어를 동일한 비율로 참고하는 것이 아니라 **해당 시점에서 예측해야 할 단어와 가장 연관이 있는 단어를 좀 더 집중(attention)해서 보겠다**는 것이다. 그리고 이는 수학적으로 디코딩시 현재 타임스텝의 출력 값(hidden state)에 가중치를 곱해 쿼리를 만들고 이를 인코더 전체의 타임스텝 출력 값과 내적(dot product)해 예측해야 하는 단어를 잘 참조할 수 있도록 역전파를 통해 해당 가중치들을 학습시키는 것이다.

유명한 **사전학습 언어모델(PLM)**들은 모두 트랜스포머 기반으로 만들어졌다. BERT는 트랜스포머의 인코더 12개로 구성되어 자연어 이해에 강점을 보이는 모델이고, GPT-1는 트랜스포머 디코더 12개 만으로 구성되어 자연어 생성에 강점이 있다. 그 뒤 언어 모델들은 모델 크기와 데이터셋을 키우며 발전해 왔는데 GPT-3는 디코더 96개에 1750억 개 파라미터를 가진 모델 버전이 가장 크다. ChatGPT는 이 GPT-3 모델을 기반으로 파인튜닝(fine-tuning)한 모델로 대화에 특화된 모델이다.

현재는 LLM 시대



LLM은 딥러닝(Deep Learning)으로 만든 파라미터가 약 1천억개가 넘는 언어 처리를 위한 모델을 말한다. 언어 처리는 언어 번역, 요약, 생성, 문장 완성, 감성 분류, 키워드 추출, 개체명 인식 같은 것들이다.

LLM에서 말하는 **파라미터**는 딥러닝에서 레이어로 데이터가 흘러 갈 때 곱셈을 해주는데 사용하는 **실수형 가중치 값**들을 의미한다. 파라미터가 약 1천억개가 넘어가게 되면 기존 언어모델은 보여주지 않았던 매우 창발적인(emergence) 답을 하고 마치 인간인 것처럼 지능적으로 답을 하는 특성을

가지게 된다.

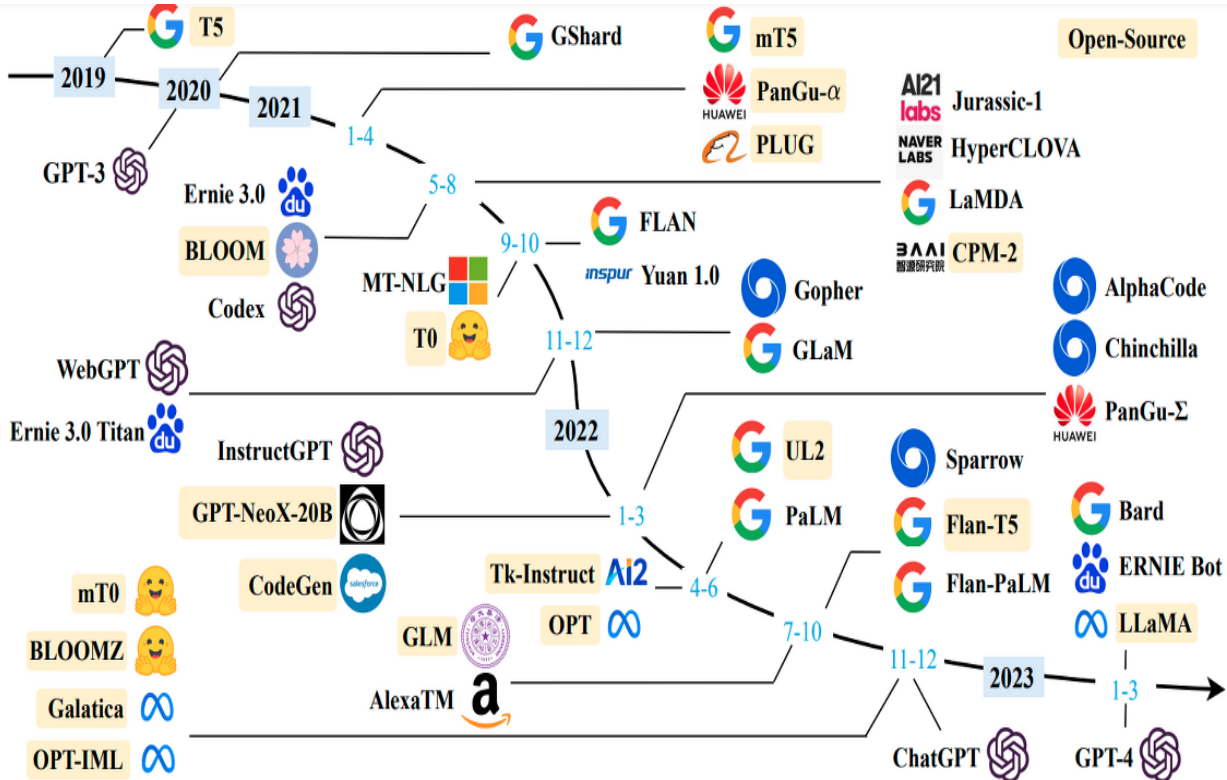


Fig. 1. A timeline of existing large language models (having a size larger than 10B) in recent years. We mark the open-source LLMs in yellow color.

[국내의 LLM]

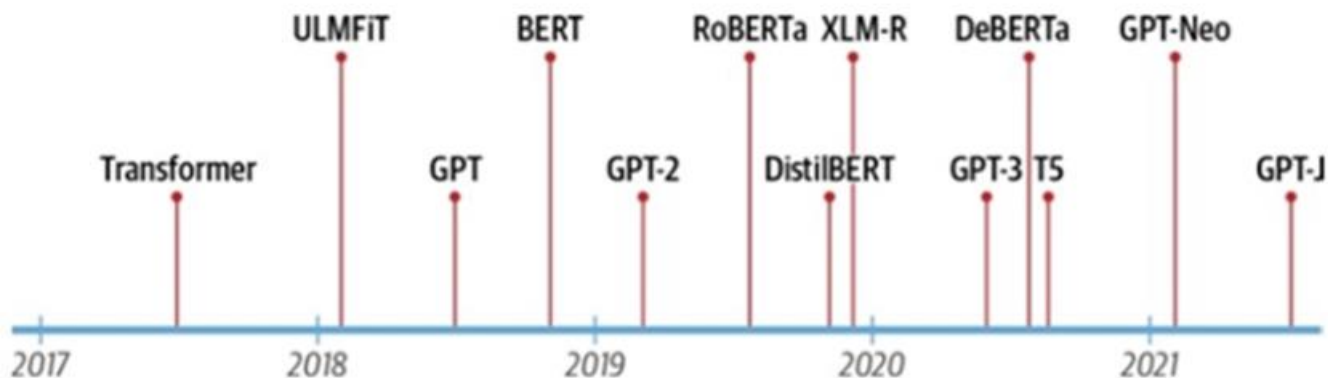
기업명	대규모 언어 모델명
네이버	하이퍼클로바 X
카카오	코GPT(koGPT)
KT	믿음(MI:DEUM)
SKT	에이닷(A.)
LG	엑사원(EXAONE)
엔씨소프트	바르코(VARCO)

BERT 와 GPT

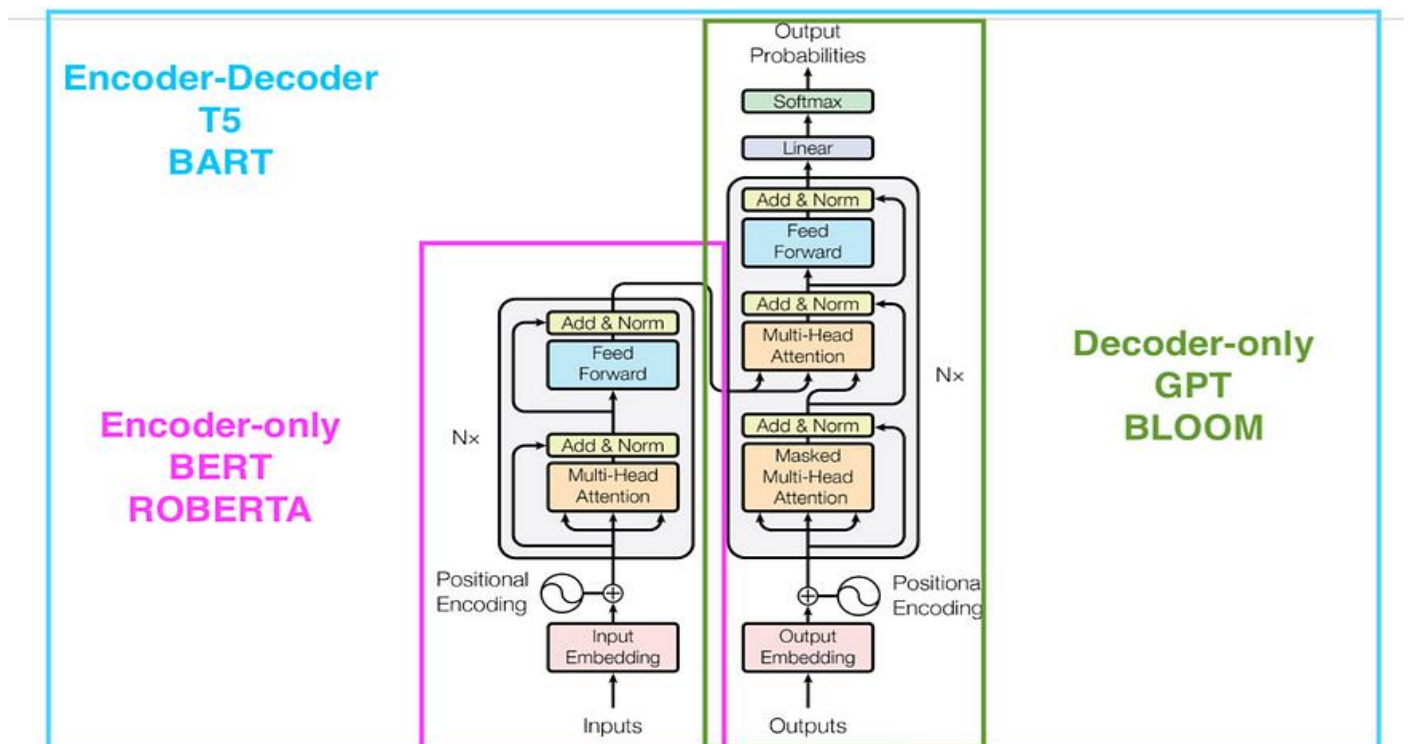
BERT(Bidirection Encoder Representations from Transformers)

GPT(Generative Pre-trained Transformer)

트랜스포머 아키텍처와 비지도 학습을 결합해 작업에 특화된 모델을 밑바닥부터 훈련할 필요를 없애고, 거의 모든 NLP 벤치마크에서 큰 차이로 기록을 경신했다. GPT와 BERT가 릴리스된 후 다양한 트랜스포머 모델이 등장했는데, 그중 주목해야 할 모델의 등장 시기는 다음 그림과 같다.



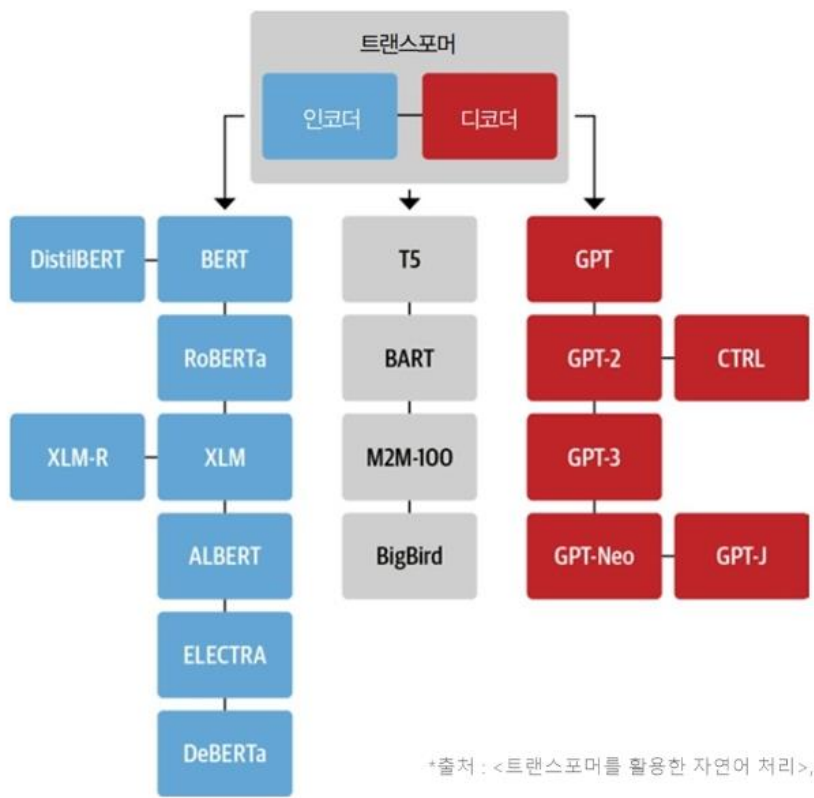
트랜스포머 모델의 주요 유형은 인코더, 디코더, 인코더-디코더 세 가지이다. 초기 트랜스포머 모델의 성공은 이후 폭발적인 모델들의 개발로 이어졌다. (disti는 경량model)



Ex. 글을 읽고, 음악이나 그림을 만들어 줄 때는 글의 뜻 파악만 하면 되니 인코딩만 - bart / 이 그림을 보고 어떤 류의 고양이인지 문장을 만들어줘 등의 디코딩만 했.

연구자들은 사전 훈련용으로 다양한 크기와 특성의 데이터셋에서 모델을 개발하고 아키텍처를 수정해 성능을 높였다. 모델의 종류가 빠르게 늘고 있지만 여전히 크게는 앞서 언급한 세 유형으로 나눌 수 있다.

트랜스포머 가계도
세 개의 주요 아키텍처는 시간이 지나면서 자생적으로 진화했다.



[인코더 유형]

트랜스포머 아키텍처를 기반으로 한 첫 번째 인코더 유형의 모델은 BERT이다. 이 모델이 공개될 당시에는 유명한 GLUE 벤치마크에서 최상의 모델을 모두 능가했다. 이 벤치마크는 난이도가 다양한 작업으로 자연어 이해(NLU)를 측정한다. 이 후에 성능을 더 향상시키기 위해 BERT의 사전 훈련 목표와 구조가 조정되었다. 인코더 유형의 모델은 여전히 연구와 산업 분야 NLU 작업에서 지배적이다. 텍스트 분류, 개체명 인식, 질문 답변 등이 이에 속한다.

- BERT

BERT의 사전 훈련 목표 두 가지는 텍스트에서 마스킹된 토큰을 예측하는 것과 한 텍스트 구절이 다른 텍스트 구절 뒤에 나올 확률을 판단하는 것이다. 전자의 작업을 마스크드 언어 모델링(MLM)이라 하고 후자를 다음 문장 예측(NSP)이라 한다.

- DistilBERT

BERT가 훌륭한 결과를 제공하지만, 모델 크기 때문에 지연시간이 짧아야 하는 환경에는 배포하기 어렵다. DistilBERT는 사전 훈련 과정에서 지식 정제(knowledge distillation)라는 기술을 사용해서 BERT보다 40% 더 적은 메모리를 사용하고 60% 더 빠르면서 BERT 성능의 97%를 달성한다.

- RoBERTa

BERT가 공개된 후 이어진 연구에서 사전 훈련 방법을 수정하면 성능이 더 향상된다는 사실이 밝혀졌다. RoBERTa는 더 많은 훈련 데이터로 더 큰 배치에서 더 오래 훈련하며 NSP 작업을 포함하지 않는다 이런 변경을 통해 원래 BERT 모델에 비해 성능이 크게 향상되었다.



Hugging Face

최근의 대부분 인공지능 연구나 프로젝트에는 딥러닝이 이용되고 있다. 비전에서만 성과를 내던 딥러닝이 최근에는 자연어 처리 분야에서도 좋은 성과를 내면서 딥러닝의 활용은 더욱 다양해 지고 있다.

딥러닝 프로그램이나 모델을 구현하여 실행하는 것은 개념적으로 상당히 복잡한 부분이 있지만 텐서플로우, 파이토치 등의 라이브러리를 사용하면 딥러닝 모델 개발을 보다 간단하게 수행할 수 있지만 딥러닝 모델을 구축하려면 상당한 비용과 시간이 모두 필요하다.

특히 최근의 LLM(Large Language Model)은 개인이나 웬만한 기업이 처음부터 구축하는 것이 사실상 거의 불가능할 정도의 비용이 들어간다. 이러한 경우, 어떤 작업 수행을 위한 모델을 개발하는 경우 기존에 다른 기업이나 사람이 개발하고 학습시킨 모델을 재사용할 수 있다면 딥러닝 등 인공지능 모델에 대한 접근이 좀 더 수월해질 수 있다.

허깅페이스는 머신러닝 기술을 제공하는 오픈소스 및 플랫폼 제공업체로서 2016년에 설립되었으며 뉴욕에 본사를 두고 있다. 텍스트 분류, 감정 분석 등과 같은 NLP 작업(Natural Language Processing)을 위해 사전 학습된 언어 모델, 이미지, 오디오 그리고 멀티 모달 등에 관련된 다양한 모델을 제공한다.

허깅 페이스

🇺🇸 15개 언어 ▼

문서 토론

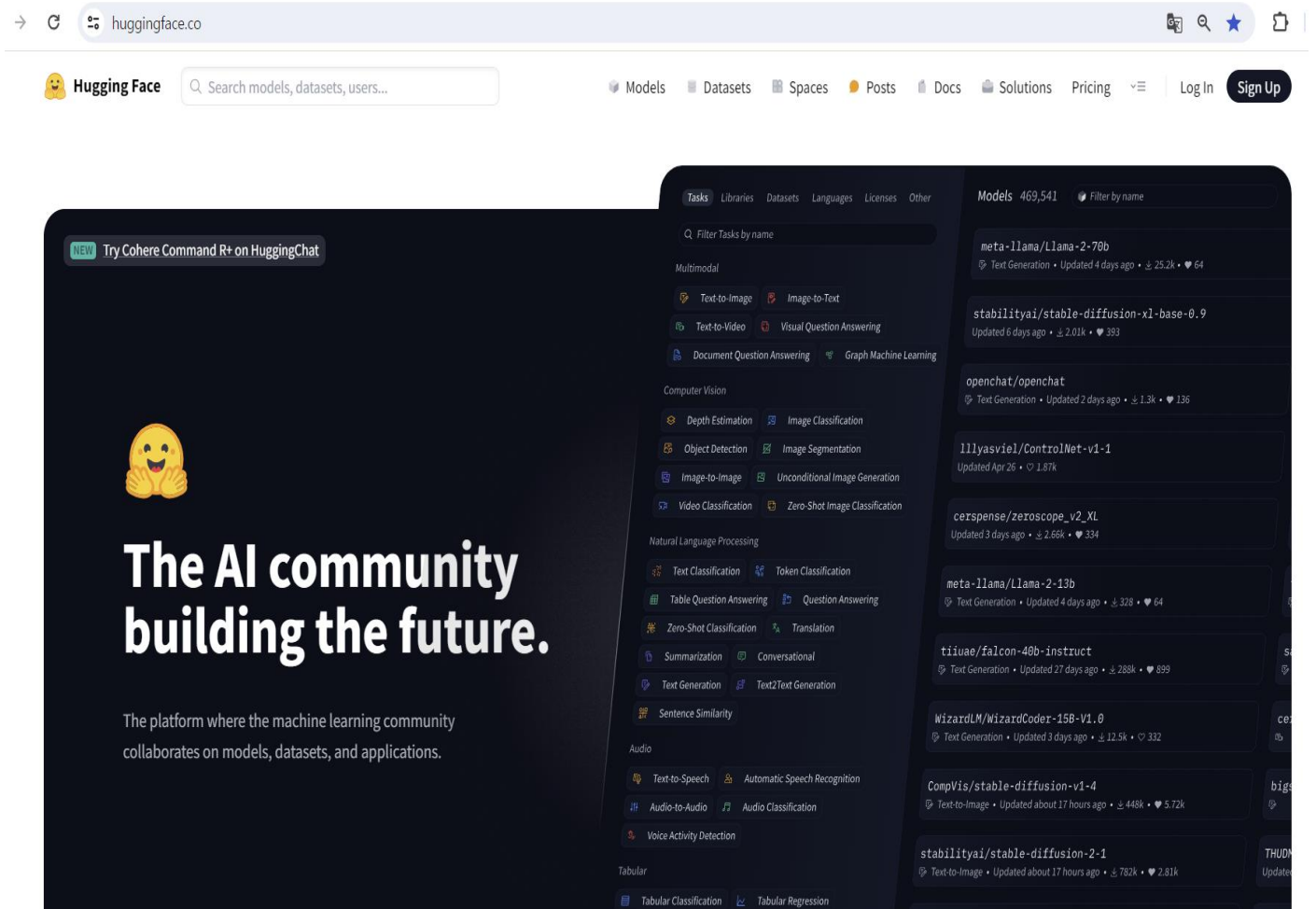
읽기 편집 역사 보기 도구 ▼

위키백과, 우리 모두의 백과사전.

허깅 페이스(Hugging Face, Inc.)는 기계 학습 모델을 구축, 배포 및 교육하기 위한 도구와 리소스를 개발하는 프랑스계 미국 회사이자 오픈 소스 커뮤니티이다. 뉴욕시에 본사를 둔 이 회사는 자연어 처리를 위해 구축된 Transformers 라이브러리와 커뮤니티 협업 및 접근성에 중점을 둔 것으로 가장 유명하다. 해당 플랫폼을 통해 사용자는 기계 학습 모델과 데이터 세트를 공유하고 작업을 선보일 수 있다.

[Hugging Face 에서 제공되고 있는 모델의 종류]

- (1) 📝 자연어 처리: 텍스트 분류, 명명된 엔티티 인식, 질문 답변, 언어 모델링, 요약, 번역, 객관식, 텍스트 생성.
- (2) 🖼️ 컴퓨터 비전: 이미지 분류, 물체 감지 및 분할.
- (3) 🗣️ 오디오: 자동 음성 인식 및 오디오 분류.
- (4) 🌸 멀티모달: 광학 문자 인식, 스캔 문서에서 정보 추출, 비디오 분류, 시각적 질문 답변.



[Hugging Face 장점]

- 가장 좋은 점은 모델을 처음부터 개발하고 훈련할 필요가 없다는 것이다. 사전 학습된 모델을 로드하여 NLP(NLP(Natural Language Processing, 자연어 처리)) 등의 작업을 쉽게 처리할 수 있다. 물론 필요에 따라서는 특정 작업에 맞게 미세 조정(파인튜닝)할 수도 있다.
- 허깅 페이스가 인기 있는 또 다른 이유는 사용 편의성이다. 간단하고 사용자 친화적인 인터페이스가 제공되어 단 몇 줄의 코드로 사용이 가능하다.
- 적극적인 허깅페이스 커뮤니티가 있어 지속적으로 발전하고 있다. 커뮤니티는 사전 학습된 모델을 공유함으로써 라이브러리의 성장에도 기여하고 있다.
- 'Models'로 공개하고 있는 학습이 끝난 모델은 'Spaces'로 간단하게 데모 애플리케이션을 작성 및 공개하는 것도 가능하다.
- 이미지 생성 이외의 음성 인식이나 분자의 3차원 구조 예측에 이르기까지 다양한 태스크에도 대응한다. 사전 학습된 모델을 사용할 수 있어 컴퓨팅 비용 절감과 시간 절감으로 이어진다.
- Hugging Face Hub에서 공개되는 음성, 이미지, 텍스트 등의 학습용 데이터셋에 액세스하기 쉬운 간단한 코드로 실행 가능한 라이브러리로, 대규모 데이터셋의 전처리를 효율적으로 실시할 수 있다.
- 자연 언어 처리에 있어서의 토큰화를 심플한 코드로 실시할 수 있는 라이브러리로, 문장이나 구의 의미를 하나의 덩어리를 나타내는 「토큰」으로 분할할 수 있다. Tokenizer 라이브러리는 다양한 토큰화 알고리즘을 간단하게 실행하고 고속으로 데이터 세트를 토큰화할 수 있다.

[Hugging Face 에 올려진 모델의 사용 방법]

- Transformers 라이브러리
 - Inference API(회원가입과 Access키가 있어야 함)
- (1) Transformers 라이브러리

Hugging Face 에서 제공되는 사전 학습 모델을 쉽게 다운로드하고 훈련할 수 있는 API 와 도구를 제공한다. 특정 모델의 동작에 필요한 전처리 및 후처리 단계를 연결하여 보다 간단하게 처리하는 **파이프라인** 방법과 **AutoClass**를 사용하여 사전 훈련된 인스턴스를 직접 로드하여 처리하는 방법으로 나뉜다.

- 파이프라인 방법 - pipeline() 함수

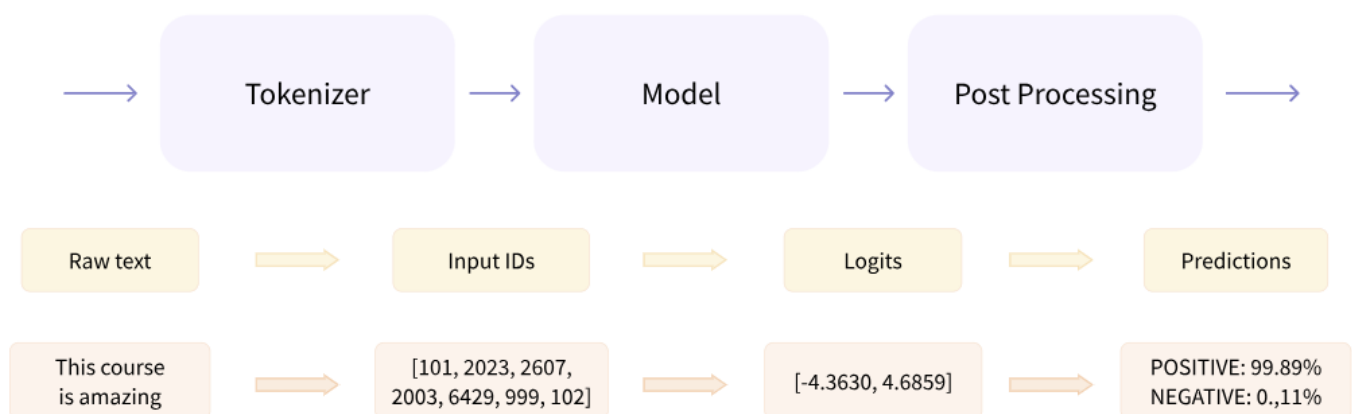
Transformers 라이브러리의 가장 기본적인 객체는 pipeline() 함수이다. 이 함수는 특정 모델과 동작에 필요한 전처리 및 후처리 단계를 연결하여 텍스트를 직접 입력하고 이해하기 쉬운 답변을 얻을 수 있다.

```
from transformers import pipeline
classifier = pipeline(task="text-classification") # sentiment-analysis 도 가능
classifier("I've been waiting for a HuggingFace course my whole life.")
[{'label': 'POSITIVE', 'score': 0.9598049521446228}]
```

여러 개의 문장을 동시에 입력으로 전달할 수도 있다.

```
classifier(["I've been waiting for a HuggingFace course my whole life.",
           "I hate this so much!"])
[{'label': 'POSITIVE', 'score': 0.9598049521446228},
 {'label': 'NEGATIVE', 'score': 0.9994558691978455}]
```

파이프라인은 **전처리(preprocessing)**, **전처리된 데이터를 모델로 전달**, **후처리(postprocessing)**의 3단계를 한번에 실행한다.



파이프라인에 텍스트가 입력되면 3가지 주요 단계가 내부적으로 실행된다.

텍스트는 모델이 이해할 수 있는 형식으로 **전처리(preprocessing)** 된다.

전처리 완료된 입력 텍스트는 **모델에 전달**된다.

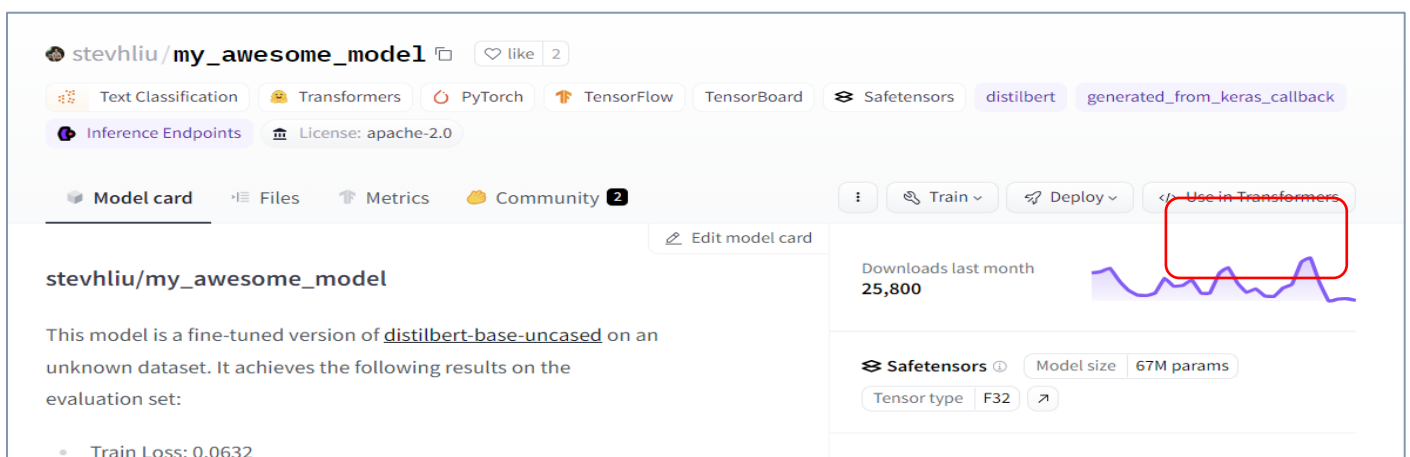
모델이 예측한 결과는 후처리 (postprocessing) 되어 우리가 이해할 수 있는 형태로 변환된다.

sentiment-analysis 파이프라인의 기본 모델은 **distilbert-base-uncased-finetuned-sst-2-english** 모델이다.

```
from transformers import pipeline
classifier = pipeline("text-classification", "distilbert-base-uncased-finetuned-sst-2-english")
classifier([ "I've been waiting for a HuggingFace course my whole life.",
            "I hate this so much!" ]
)
```

```
classifier = pipeline(task="text-classification",
                      model="stevhliu/my_awesome_model")
classifier(text)
```

```
classifier = pipeline(model="facebook/bart-large-mnli")
classifier( # 얼마나 비중이 있는 단어인지
            "I have a problem with my iphone that needs to be resolved asap!!",
            candidate_labels=["urgent", "not urgent", "phone", "tablet", "computer"],
        )
```



task 는 리턴되는 파이프라인의 종류를 결정하며 다음과 같은 task 들을 지정할 수 있다.(task명)

"audio-classification": will return a AudioClassificationPipeline.
"automatic-speech-recognition": will return a AutomaticSpeechRecognitionPipeline.
"conversational": will return a ConversationalPipeline.
"depth-estimation": will return a DepthEstimationPipeline.
"document-question-answering": will return a DocumentQuestionAnsweringPipeline.
"feature-extraction": will return a FeatureExtractionPipeline.
"fill-mask": will return a FillMaskPipeline. (빈칸에 들어갈 단어 찾아봐)
"image-classification": will return a ImageClassificationPipeline.
"image-feature-extraction": will return an ImageFeatureExtractionPipeline.
"image-segmentation": will return a ImageSegmentationPipeline.
"image-to-image": will return a ImageToImagePipeline.
"image-to-text": will return a ImageToTextPipeline.
"mask-generation": will return a MaskGenerationPipeline.
"object-detection": will return a ObjectDetectionPipeline.
"question-answering": will return a QuestionAnsweringPipeline.
"summarization": will return a SummarizationPipeline.
"table-question-answering": will return a TableQuestionAnsweringPipeline.
"text2text-generation": will return a Text2TextGenerationPipeline.
"text-classification" (alias "sentiment-analysis" available):
will return a TextClassificationPipeline.
"text-generation": will return a TextGenerationPipeline:.
"text-to-audio" (alias "text-to-speech" available): will return a TextToAudioPipeline:.
"token-classification" (alias "ner" available): will return a TokenClassificationPipeline.
"translation": will return a TranslationPipeline.
"translation_xx_to_yy": will return a TranslationPipeline.
"video-classification": will return a VideoClassificationPipeline.
"visual-question-answering": will return a VisualQuestionAnsweringPipeline.
"zero-shot-classification": will return a ZeroShotClassificationPipeline.
"zero-shot-image-classification": will return a ZeroShotImageClassificationPipeline.
"zero-shot-audio-classification": will return a ZeroShotAudioClassificationPipeline.
"zero-shot-object-detection": will return a ZeroShotObjectDetectionPipeline.

```
conda install pytorch torchvision -c pytorch # 파이토치
pip install sympy==1.13.1 fsspec
pip install transformers
pip install sentencepiece
pip install hf_xet
pip install pillow
pip install sacremoses
```